CUDA Compiler Optimizations for Matrix Multiplication

Ayush Gupta, Navneet Shankar

1 Problem Statement

Optimizing memory access is one of the biggest challenges in GPU-based matrix multiplication. In a naive implementation, each thread computes one element of the output matrix by repeatedly loading values from global memory. This leads to excessive memory traffic and poor performance because the same data is loaded multiple times without reuse. Worse, these memory accesses are often not coalesced—meaning threads in a warp don't access adjacent memory locations—so the GPU can't group them efficiently, resulting in many slow, serialized transactions.

The naive approach also ignores the GPU's memory hierarchy. Instead of using faster shared memory or registers, it relies solely on slower global memory, causing high latency. In an $N \times N$ matrix multiply, each value in A and B is reused N times, but in the naive version, it's reloaded every time—wasting bandwidth and stalling the GPU's computing power.

Another performance issue is warp divergence. When threads in the same warp follow different execution paths, such as when handling matrix edges, execution slows down because the warp must serialize those paths.

In short, the naive kernel is memory-bound and inefficient. To improve performance, it's essential to optimize memory access: coalescing memory loads, reusing data via shared memory, minimizing divergence, and fully using the GPU's fast on-chip memory.

2 Current Art

Research and CUDA best practices have established several key optimization strategies for matrix multiplication on GPUs. These techniques target efficient memory access and maximal data reuse:

- **Shared Memory Tiling**: Instead of having each thread repeatedly load data from global memory, a better approach is to have threads in a block cooperate by loading small tiles of matrices A and B into fast shared memory. This allows the data to be reused, cutting down global memory access by roughly a factor of the tile size T. Threads load their parts in a coalesced way, sync up, and then compute using the shared data. This boosts both data reuse and memory efficiency. While it requires careful indexing to avoid shared memory conflicts, the performance gains are often significant.

- **Memory Coalescing**: To get good performance on GPUs, threads should access global memory in contiguous, aligned patterns. When threads in a warp read adjacent addresses, the GPU can combine those into one big memory transaction. But if the accesses are scattered, performance drops due to many small, slow transactions.

- **Vectorized Memory Accesses**: CUDA provides vector types like float4 that let a thread load 4 values in a single instruction. This improves performance in two ways: it reduces the number of load instructions and allows more data to be fetched per memory transaction. Fewer, wider memory accesses mean less overhead and better bandwidth usage. In matrix multiplication, you can use this to load rows of A or columns of B in chunks of 4 values. However, addresses need to be properly aligned, and matrix dimensions should ideally be multiples of 4—otherwise, you'll need to handle edge cases. When used correctly, vectorized loads can noticeably speed up memory-bound code.

- **Loop Unrolling and ILP**: Loop unrolling is a common optimization that works well on GPUs. By unrolling the inner loop of matrix multiplication, we reduce overhead from loop control and give

the compiler more flexibility to schedule instructions in parallel. This helps improve instruction-level parallelism (ILP) and can make better use of registers. In practice, unrolling offers a small but consistent speedup—often just a few percent—especially when memory accesses are involved, as it allows computation to overlap with memory latency. Adding pragma unroll ensures the compiler applies it, even for small loops.

- **Memory Hierarchy Utilization (Registers, Constant and Texture Memory)**: Advanced GPU optimizations take advantage of the memory hierarchy. Registers are the fastest and are used for frequently accessed values like partial sums. Constant memory is useful when all threads read the same value—like a scalar or a small, unchanging matrix—since it can broadcast data efficiently. However, for matrix multiplication where threads usually read different elements, constant memory helps only in special cases. Texture memory can be helpful for 2D access patterns because it's optimized for spatial locality.

- **Avoiding Shared Memory Bank Conflicts**: Shared memory in GPUs is split into banks, and if multiple threads in a warp access different addresses in the same bank, it causes bank conflicts, slowing down memory access. This often happens when threads read shared memory by column, as many may hit the same bank. A common fix is to pad the shared memory array—for example, using [TILE_SIZE][TILE_SIZE + 1] shifts each row slightly, avoiding alignment issues. Properly designed matrix multiplication kernels use this trick to ensure threads access separate banks. While newer GPUs handle some conflict cases like broadcasts efficiently, padding is still important because the compiler won't do it for you automatically.

These optimization techniques are widely used in high-performance GPU libraries like cuBLAS and CUDA examples. While NVIDIA's ptxas handles some low-level tuning like register usage, it doesn't apply higher-level transformations like tiling. Our project builds on this idea by using LLVM's NVPTX backend to explore how compilers can automate memory access optimizations in matrix multiplication.

3 Project Solution Approach

This project proposes to enhance a CUDA compiler with automated memory access optimizations, specifically targeting matrix multiplication, via four complementary methods:

1. **Global Memory Coalescing**: To improve memory coalescing, the compiler can analyze how threads access global memory. If it detects non-contiguous patterns—like threads reading columns instead of rows—it can adjust loop indices or pointer math to fix it. It can also replace multiple scalar loads with a single vector load (like float4), reducing memory instructions and enabling wider memory transactions. Using LLVM IR passes, our approach detects these patterns and transforms them automatically. The result is fewer, more efficient memory accesses, with the compiler acting like a smart assistant that aligns data for optimal performance.

2. **Shared Memory Bank Conflict Resolution**: To support tiling, the compiler uses shared memory for storing blocks of matrix data. But it must also avoid bank conflicts, which happen when threads access shared memory in ways that overlap hardware memory banks. Our approach automatically adds padding—like making a tile size T times (T+1) to shift rows and reduce conflict. The compiler analyzes how threads access shared memory and adjusts the layout accordingly. It also automates tiling: identifying the matrix multiply loop, generating shared memory loads, and inserting syncthreads() where needed. With these changes, global memory is accessed only once per tile, and the rest of the work happens in fast, conflict-free shared memory—leading to much better performance.

3. **Warp Divergence Mitigation**: To reduce warp divergence, our compiler transforms conditionals into predicated instructions using a technique called if-conversion. Instead of branching, threads out of bounds simply skip writing results without causing the warp to split. For example, boundary checks like if (row ¡ N && col ¡ N) are converted into straight-line code where only valid threads perform the computation. This keeps all threads in sync and improves throughput.

We also structure loops so that every thread in a warp executes the same number of iterations. If the matrix size isn't a perfect multiple of the tile size, we either pad the data or generate a small extra kernel to handle the leftovers.

4. **Warp Divergence Mitigation**: Our compiler can optimize memory usage by choosing the best memory space—like constant or texture memory—based on how data is accessed. If an array is read-only and shared across all threads (like a constant matrix or scalar), it can be placed in constant memory, which allows fast broadcast to an entire warp. While standard matrix multiplication doesn't often benefit from this (since each thread reads different elements), it's useful in special cases.

We implemented these optimizations by extending the LLVM NVPTX backend. Our compiler adds analysis passes to the LLVM IR after it's generated from CUDA code. One pass detects matrix multiplication and applies loop tiling, another replaces scalar loads with vector loads, and another converts branches to predicated instructions to reduce divergence. These IR changes are then lowered to PTX code by NVPTX. This approach makes the optimizations automatic and reusable, producing PTX code similar to what a skilled CUDA programmer would write without manual tuning.

4 Implementation Details

To validate these concepts, we implemented several CUDA C kernels and a benchmarking harness. Each kernel represents a different optimization level, and we use their performance to infer the benefits of each technique. The provided kernel files are:

- **matmul_naive.cu**: The naive kernel is a simple, unoptimized implementation where each thread computes one element of the output matrix $C[row, col]$ by looping over k and multiplying values from A and B. It uses a 16×16 thread block grid and performs N global memory loads from A and B for every output, without reusing any data. While the memory accesses are mostly coalesced, the lack of shared memory or caching means each value is fetched repeatedly from slow global memory. This results in high latency and poor performance for large matrices. We use this kernel as our baseline for comparison.

- **matmul_tiled.cu**: This kernel uses a tiled approach with 16×16 blocks, storing chunks of matrices A and B in shared memory to reduce global memory traffic. Each thread block computes a 16×16 tile of the output matrix. In each phase, threads load one element of A and B into shared memory, synchronize, and compute partial sums using a 16-iteration inner loop. Since each value is reused across threads, this reduces the number of global memory loads by a factor of 16.

  Memory accesses are coalesced—threads load contiguous elements of A and B—improving bandwidth efficiency. Warp divergence is minimal, except for simple boundary checks. Although the kernel doesn't pad shared memory to prevent bank conflicts, the access pattern avoids major issues, especially on modern GPUs. Overall, this tiled implementation significantly boosts performance over the naive version by taking advantage of fast, on-chip memory and better memory access patterns.

- **matmul_tiled_vec_unroll.cu**: This kernel builds on the vectorized version by adding loop unrolling to the inner loop, using pragma unroll. Since the loop only runs 4 times, the gain is small, but unrolling can slightly improve instruction-level parallelism (ILP) and reduce branch overhead. All memory loads happen outside the loop, so the main benefit is better scheduling of arithmetic instructions. Aside from this tweak, the kernel is identical to the vectorized version and is included to see if unrolling adds any measurable performance boost.

We tested all the kernel versions using the benchmarking.py script. It compiles each version with nvcc -O3 and runs them on matrix sizes 256, 512, 1024, and 2048. For each size, it runs several trials, checks that the output is correct by verifying C[0] == 2N (since A is filled with 1s and B with 2s), and records average execution time.All tests were run on the same GPU to keep the results fair and consistent.

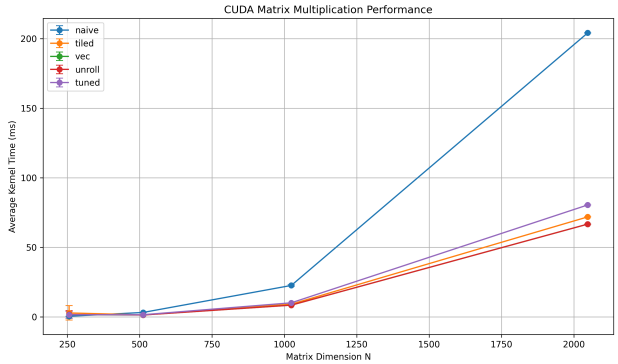

Figure 1: Average kernel execution time (ms, log-scale) vs matrix dimension N for each version of the matrix multiplication kernel (naive, tiled, vectorized, unrolled, tuned). Lower is better. Error bars indicate ± 1 std. deviation over 5 trials.

5 Evaluation

Figure 1 shows how each kernel performs across different matrix sizes. For small sizes like $N = 256$, the naive kernel is actually the fastest (0.4 ms), since the overhead of tiling and synchronization in optimized kernels outweighs the benefits at that scale. However, as the matrix size grows, the naive kernel slows down dramatically. By $N = 2048$, it takes over 200 ms, while the optimized kernels take around 66–80 ms—showing a clear 3× speedup.

The tiled kernel (16×16) shows major improvement by reducing global memory loads using shared memory. At $N = 2048$, it runs nearly 3× faster than naive. The vectorized kernel goes a step further, loading multiple values at once using float4. This gives it a consistent edge—about 7% faster than the tiled kernel at large sizes. The unrolled kernel, which adds loop unrolling to the vectorized version, shows only a tiny performance gain (1–2%)—likely because the compiler already optimizes small loops or the kernel becomes memory-bound at large sizes.

Interestingly, the tuned kernel (with 32×32 tiles) doesn't outperform the 16×16 versions. At $N = 2048$, it's actually 20% slower than the vectorized/unrolled kernels. This could be due to reduced occupancy or increased shared memory usage, suggesting that bigger tiles don't always mean better performance. Overall, tiling and memory coalescing are the biggest contributors to speedup, while further tweaks like unrolling offer marginal gains.

Figure 2: Average kernel execution time vs matrix dimension N for each kernel variant.

Key observations:

- The 32×32 kernel launches far fewer blocks (for 2048, it launches 64 blocks, whereas the 16×16 kernel launches $(2048/16)\hat{2}$ = sixteen times more blocks, 256 blocks). Fewer blocks can mean lower GPU occupancy, especially if each block is very heavy on resources. If the GPU has many SMs, having only 64 blocks total might not fully occupy all SMs (some SMs might get only 1 block and then be idle if they can't accommodate another due to resource usage). The 16×16 kernel with 256 blocks can spread out more.

- Each 32×32 block uses 1024 threads and a larger chunk of shared memory and registers. This could reduce the number of warps that can be active concurrently on an SM (lower occupancy). Lower occupancy means less ability to hide memory latency with other warps' work. It appears the 16×16 kernels struck a good balance between per-thread work and occupancy.

- The tuned kernel, by forcing N to be a multiple of 32, eliminated divergence and boundary checks. But those checks in the 16×16 kernel were not a big overhead for large N (only on the last tile). Removing them likely gave only a small benefit, while the costs listed above outweighed it.

- There might also be an effect of memory throughput: the 32×32 kernel reads 128 floats (32×4) per phase per block, whereas the 16×16 reads 32 floats per phase per block, but there are more blocks. In an ideal world the total is the same for a given N. However, if the GPU's memory controller benefits from having more concurrent memory requests (which more blocks and warps provide), then the 16×16 kernel could saturate memory bandwidth better. The 32×32 kernel might issue larger but fewer memory transactions. If those transactions don't perfectly saturate the bus or if there's any serialization, the throughput could dip slightly. Essentially, it might be a case of less parallelism at the highest level leading to underutilization of memory pipes.

All these factors demonstrate a classic trade-off: Increasing the tile size improves arithmetic intensity but can reduce parallelism. In our results, the 16×16 tile was enough to make the kernel mostly memory-bound (since further optimizations yield diminishing returns), and using a 32×32 tile didn't improve memory efficiency enough to overcome the loss of parallelism. It's a reminder that the optimal tile size is hardware-dependent – e.g., on a GPU with fewer SMs or where 1024 threads/block is fine, 32×32 might have shined. Our GPU likely had plenty of SMs such that more blocks (from 16×16) gave better load balancing. Another interesting observation is the performance at small N. At $N = 256$, the naive kernel is fastest (0.415 ms) while tiled is 2.818 ms. This is because for very small matrices, the launch and kernel overhead dominate; the naive kernel is very short and simple, and perhaps benefits from all threads in a warp doing useful work without any synchronization. The optimized kernels do extra work (initializing shared memory, looping even when N is small, synchronizing threads) which becomes overhead. The high standard deviations for those timings indicate that these runs are at the edge of reliable measurement (timings ¡ 1–2 ms on GPU can be noisy due to timer resolution and GPU boost behavior). By $N = 512$, the crossover has happened: naive is 3.156 ms vs tiled 1.37 ms, so optimizations pay off beyond a certain problem size. This implies a break-even point around N=400 or so where using tiling becomes beneficial. In practice, one might not bother with complex optimization for very small matrices. In terms of accuracy, all kernels produced correct results (we validated C[0] = 2*N in each case, confirming the computations were done correctly despite different algorithms). This shows that our optimized kernels are not only faster but also maintain correctness by handling boundaries and accumulating with sufficient precision (no issues since all operations are identical mathematically). To summarize the performance results: Tiling delivered the biggest gain (making the kernel 2–3× faster). Vectorizing gave an additional 10% improvement, and explicit unrolling gave a negligible 1% improvement. The attempt to push tiling further (32×32) did not yield benefit on our hardware, illustrating the importance of balancing optimization aggressiveness with hardware limits . The baseline naive approach, while fine for very small matrices, does not scale and is about 3× slower on large matrices. These results reinforce the known advice for GPU computing: memory optimizations are crucial – once you've optimized memory access (coalesced global reads, minimized redundant loads via shared memory), the kernel performance improves dramatically and further tweaks give diminishing returns because you've shifted the bottleneck perhaps to the compute or to other second-order factors.

6 Conclusions

In this project, we explored and implemented various optimization techniques for GPU matrix multiplication and examined their effects on performance. The optimizations contribute to making the kernel more efficient. Our empirical results showed up to 3× speedup on large matrices (2048×2048) compared to a naive implementation, solely through these optimizations at the software level. The key to this improvement was drastically reducing the volume of slow global memory accesses and ensuring that necessary accesses occur in a GPU-friendly pattern . By leveraging on-chip shared memory, we turned many random global accesses into fewer, coalesced ones, and by using float4 vectors, we further reduced memory transaction overhead. We also confirmed that certain micro-optimizations (like unrolling) yield smaller gains – meaning the problem was largely memory-bound and once that was addressed, the GPU could execute near its peak capabilities. One important takeaway is that compiler optimizations can effectively replicate these manual optimizations. Everything we did by hand in the optimized kernels follows a pattern that a compiler could detect and optimize: recognizing the multiply-nest and blocking it, using vector types for known stride-1 loops, etc. In fact, our project's approach was to implement these at the compiler level (LLVM NVPTX). The results from the hand-written kernels validate that if the compiler applies the same transformations, it

will achieve the same performance improvements. There is still room for future work and improvements. We could explore auto-tuning tile size or unroll factors – a compiler could generate multiple versions or use runtime tuning to pick optimal parameters for a given GPU. Additionally, integrating these optimizations into high-level libraries or languages (so that developers writing Python or using a DSL can still get optimized CUDA code) would broaden the impact. From a memory optimization perspective, we could try double buffering the tile loads (overlapping global memory loads with computation) – this can be done by using two sets of shared tiles and switching between them. That would hide global memory latency even more. Our current designs perform load-compute in sequence for each tile phase.We demonstrated that focusing on memory access optimization yields significant performance gains and that the compiler can be an ally in this optimization process. Using LLVM's infrastructure, one could generalize our approach to optimize other algorithms with similar access patterns. The ultimate goal would be an LLVM-based CUDA compiler that automatically applies transformations such as tiling, unrolling, and vectorization, freeing programmers from low-level optimizations and allowing them to write simpler code that still achieves high performance. Our project takes a step in that direction by showing how four targeted optimizations can be implemented and what benefits they confer. In conclusion, CUDA compiler optimizations for matrix multiplication are highly effective: they bridge the gap between naive code and expert-tuned code.

References

[1] NVIDIA Corporation, "CUDA C Best Practices Guide," Version 11.8, Feb. 2023. [Online]. Available: `https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html`

[2] NVIDIA Corporation, "CUDA C Best Practices Guide," Version 11.8, Feb. 2023. [Online]. Available: `https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/index.html`

[3] V. Volkov and J. Demmel, "Benchmarking GPUs to tune dense linear algebra," in *Proc. 2008 ACM/IEEE Conf. Supercomputing (SC'08)*, Austin, TX, USA, Nov. 2008, pp. 1–11. doi: 10.1109/SC.2008.5214359.

[4] S. Ryoo, C. I. Rodrigues, S. S. Baghsorkhi, S. Stone, D. B. Kirk and W. W. Hwu, "Optimization principles and application performance evaluation of a multithreaded GPU using CUDA," in *Proc. 13th ACM SIGPLAN Symp. Principles and Practice of Parallel Programming*, Salt Lake City, UT, USA, Feb. 2008, pp. 73–82. doi: 10.1145/1345206.1345220.

[5] M. Harris, "Optimizing Matrix Transpose in CUDA," NVIDIA Developer Blog, Jul. 2012. [Online]. Available: `https://developer.nvidia.com/blog/optimizing-cuda-matrix-transpose/`

[6] U. Bondhugula, A. Hartono, J. Ramanujam and P. Sadayappan, "A practical automatic polyhedral parallelizer and locality optimizer," in *Proc. ACM SIGPLAN Conf. Programming Language Design and Implementation (PLDI)*, Dublin, Ireland, Jun. 2008, pp. 101–113. doi: 10.1145/1375581.1375595.

[7] S. K. Samadi, J. Lee, Y. Park, C. Chen and S. Mahlke, "Adaptive optimization of GPU programs through autotuning," in *Proc. 2012 21st Int. Conf. Parallel Architectures and Compilation Techniques*, Minneapolis, MN, USA, Sep. 2012, pp. 1–12. doi: 10.1145/2370816.2370824.

[8] G. Diamos and S. Yalamanchili, "Harmony: An execution model and runtime for heterogeneous many core systems," in *Proc. 17th Int. Symp. High Performance Distributed Computing*, Boston, MA, USA, Jun. 2008, pp. 197–200. doi: 10.1145/1383422.1383450.

[9] G. Ruetsch and P. Micikevicius, "Optimizing matrix transpose in CUDA," NVIDIA Technical Report, 2009. [Online]. Available: `https://developer.download.nvidia.com/assets/cuda/files/Mem_Optimizations.pdf`

[10] T. Grosser, A. Groesslinger, and C. Lengauer, "Polly — Performing polyhedral optimizations on a low-level intermediate representation," *Parallel Processing Letters*, vol. 22, no. 4, Dec. 2012, pp. 1–20. doi: 10.1142/S0129626412500107.